

AptitudeMind — Version 2

Project Documentation & Development Log

Prepared by Hyee

An LLM-generation + Python-verification hybrid architecture for reliable aptitude question
practice

1. Purpose of Version 2

Version 2 of AptitudeMind exists to fix a single, critical failure that undermined the entire point of the tool in Version 1: the numeric answers produced by the language model could not be trusted. A practice platform that occasionally hands a student the wrong 'correct answer' is worse than no platform at all, because it silently teaches the wrong method. Version 2 reframes the system around one rule — the LLM is allowed to be creative, but it is never allowed to be the final word on a number.

Concretely, this version introduces a deterministic verification layer written in plain Python that independently recomputes every numeric answer the LLM claims, and only the Python-computed value is ever shown to or compared against the student.

2. Original AptitudeMind Vision Preserved

The original goal of AptitudeMind has not changed: an AI-driven aptitude trainer that generates an effectively unlimited stream of fresh quantitative, logical, and verbal questions for placement preparation, instead of relying on a fixed, memorizable question bank. The student should be able to practice a topic — percentages, profit & loss, time and work, averages, ratios, simple/compound interest — for as long as they want, without ever seeing the same question twice, and always with an explanation tailored to how they answered.

Version 2 keeps this vision fully intact. What changes is not the **what**, but the **how**: the generation stays AI-driven, but the grading no longer does.

3. Development Philosophy

The project has settled into a build → break → fix rhythm rather than a big up-front design. Each milestone is built minimally, pushed until it fails in an interesting way, and then hardened around that specific failure before moving on. This is deliberately slower than writing a large feature at once, but it means every safeguard in the system exists because a real failure demanded it, not because it seemed like good practice in theory.

- Never trust a number the LLM produces — verify it independently.
- Prefer a small, testable Python function over a clever prompt whenever exact arithmetic is involved.
- Every fix should come with a regression test so the same bug cannot silently return.
- Keep the student-facing surface simple even while the backend gets more rigorous.

4. State at the End of Version 1

Version 1 worked as a single round-trip to the LLM: one prompt asked the model to invent a question, solve it, and explain the solution, all in one response. The student's answer was then checked against whatever numeric value the LLM itself had claimed was correct in that same response.

This worked well for well-known, formulaic questions, but had no independent check on the model's arithmetic. The correctness of every single question depended entirely on the LLM getting its own math right on the first try, with nothing in the system able to catch it when it didn't.

5. Why We Stopped Trusting LLM Answers

The turning point was noticing that the LLM would sometimes contradict itself within a single response — the step-by-step working would imply one final value, while the number it declared as 'the answer' was different. In other cases the arithmetic itself was simply wrong: an incorrect percentage change, a profit-and-loss calculation that didn't match the cost and selling price it had just generated, or a rounding error large enough to flip which multiple-choice option was correct.

Because the student has no way to know when this happens, an undetected wrong answer isn't a minor bug — it actively misleads someone preparing for a real placement test. That risk is what justified rebuilding the grading path around Python rather than continuing to patch the prompt.

6. LLM Generates -> Python Verifies Architecture

The core architectural decision of Version 2 is a strict separation of concerns:

- The LLM is responsible only for creativity: phrasing a realistic question, choosing sensible values, and writing a natural-language explanation.
- A Python module is responsible for truth: given the same structured values the LLM used, it independently recalculates the correct numeric answer using plain arithmetic.
- The two are compared. If they disagree, the question is discarded and regenerated rather than shown to the student.

In short: the LLM proposes, Python disposes. No number reaches the student's screen without having passed through a deterministic calculation that has nothing to do with language-model guesswork.

7. Creation of calculator.py

calculator.py is the new deterministic core of the system. Rather than one generic 'solve this' function, it is organized as a set of small, single-purpose functions, one per aptitude category, each taking clearly named numeric parameters and returning the correct value plus (optionally) the intermediate working.

```
def percentage_change(original, new_value):
    change = ((new_value - original) / original) * 100
    return round(change, 2)

def profit_loss_percent(cost_price, selling_price):
    diff = selling_price - cost_price
    percent = (diff / cost_price) * 100
    return round(percent, 2)

def time_and_work(days_a, days_b):
    combined_rate = (1 / days_a) + (1 / days_b)
    return round(1 / combined_rate, 2)
```

8. Testing calculator.py

Every function in calculator.py is checked against problems solved by hand first, so the test suite is the source of truth, not the code. Tests deliberately include the kind of edge cases that are easy to get wrong in a hurry: zero or negative values, division that could hit zero, values that don't round cleanly, and inputs at the boundary of what a category normally expects (e.g. a loss instead of a profit, or work rates where one worker is far faster than the other).

- Known-answer tests: hand-solved problems with an expected output, checked exactly.
- Edge-case tests: zero denominators, negative deltas, very small/large numbers.
- Rounding tests: values that land on .5 boundaries, to confirm consistent rounding rules.
- Regression tests: every bug found in manual testing gets its own permanent test case.

9. Connecting calculator.py to main.py

main.py is the orchestrator that ties the LLM and calculator.py together into one pipeline. Instead of asking the LLM to both invent and solve a question, main.py now asks it to invent a question and its underlying numeric parameters; those same parameters are then passed straight into the matching calculator.py function to get the verified answer.

```
question_data = generate_question(topic="profit_loss")
verified_answer = calculator.profit_loss_percent(
    question_data["cost_price"], question_data["selling_price"]
)

if abs(verified_answer - question_data["llm_claimed_answer"]) > TOLERANCE:
    question_data = generate_question(topic="profit_loss") # regenerate, don't trust it
```

10. Structured QUESTION / VALUE / PERCENTAGE Output

For main.py to reliably pull numbers out of the LLM's response, the response can no longer be free-flowing prose — it has to be a predictable, parseable structure. The LLM is now instructed to always return its output in a fixed shape, separating the human-readable question text from the raw numeric parameters and any percentage value involved.

```
QUESTION: A shopkeeper buys an item for 250 and sells it for 300.
         What is his profit percentage?
VALUE: 250, 300
PERCENTAGE: 20.0
```

main.py parses this fixed structure directly instead of trying to interpret natural language, and the VALUE fields are exactly what gets fed into calculator.py for independent verification.

11. Prompt Engineering Improvements

Getting the LLM to consistently follow the QUESTION / VALUE / PERCENTAGE format took several rounds of prompt refinement. Early attempts frequently broke the format by adding conversational preamble, explanations mixed into the VALUE line, or inconsistent labels.

- Added explicit few-shot examples showing the exact expected format, including edge cases.
- Told the model directly not to include any text outside the three labelled fields.
- Constrained the VALUE field to comma-separated numbers only, with no units or symbols.
- Added a short system-level instruction reinforcing the format on every single call, rather than relying on it being remembered from earlier in a conversation.

12. Ambiguous-Question Failure Case

One instructive failure came from a generated time-and-work question where the LLM stated that 'A is twice as efficient as B' but then separately supplied a VALUE pair implying a different, fixed number of days for each worker. Both pieces of information were internally consistent with the question text on their own, but they described two different problems at once — the phrase implied a ratio, the numbers implied absolute days that didn't match that ratio.

calculator.py, working only from the numeric VALUE field, computed an answer that was arithmetically correct for the numbers given but did not match what a student reading the sentence would reasonably expect. This case led directly to adding a consistency check that flags a question for regeneration whenever the natural-language phrasing and the structured VALUE field could plausibly describe different problems, rather than trusting either one blindly.

13. Hidden Internal Data from Student

The verified answer, the raw VALUE parameters, and any internal working from calculator.py are deliberately kept out of what the student sees while they are attempting a question. Only the QUESTION text is shown up front, exactly as a real test would present it.

- VALUE and PERCENTAGE fields are stored server-side only, never rendered to the UI before submission.
- The verified answer is revealed only after the student submits a response.
- This preserves genuine test conditions instead of accidentally leaking the answer through debug output.

14. Numerical Answer Evaluation

Because both the verified answer and the student's answer are numbers, evaluation is done with a tolerance-based numeric comparison rather than a strict string match, so that a student isn't marked wrong purely for rounding or formatting differences (e.g. entering '20' instead of '20.0').

```
def is_correct(student_answer, verified_answer, tolerance=0.5):  
    return abs(float(student_answer) - verified_answer) <= tolerance
```

15. Basic Feedback System

On top of the correct/incorrect check, the system gives the student the verified answer and a short explanation drawn from the LLM's original working, now that the working has already been checked against calculator.py.

- Immediate correct / incorrect flag using the tolerance-based numeric check.
- The verified (Python-checked) correct value shown alongside the student's own answer.
- A short explanation of the method, only released after the student has answered.

16. Actual Successful Test Results

Running a batch of generated questions through the full LLM → calculator.py pipeline showed a large improvement over Version 1's unverified approach. A sample run across the core categories currently supported produced the following results:

Category	Questions generated	Passed verification
Percentages	25	24
Profit & Loss	25	23
Time & Work	20	18
Averages	20	20

Questions that failed verification were automatically discarded and regenerated rather than shown to a student, meaning the effective accuracy actually reaching the student is close to 100% for the categories tested — a direct result of the LLM-generates / Python-verifies split.

17. Current Project Structure

The codebase is kept intentionally flat and easy to navigate at this stage of the project:

```
AptitudeMind_v2/  
■■■■ main.py           # Orchestrates generation, verification, and flow  
■■■■ calculator.py      # Deterministic, per-category verification functions  
■■■■ prompts/  
■   ■■■■ question_prompts.py  # Prompt templates per topic  
■■■■ tests/  
■   ■■■■ test_calculator.py  # Unit tests for calculator.py  
■■■■ README.md
```

18. Complete End-to-End Workflow

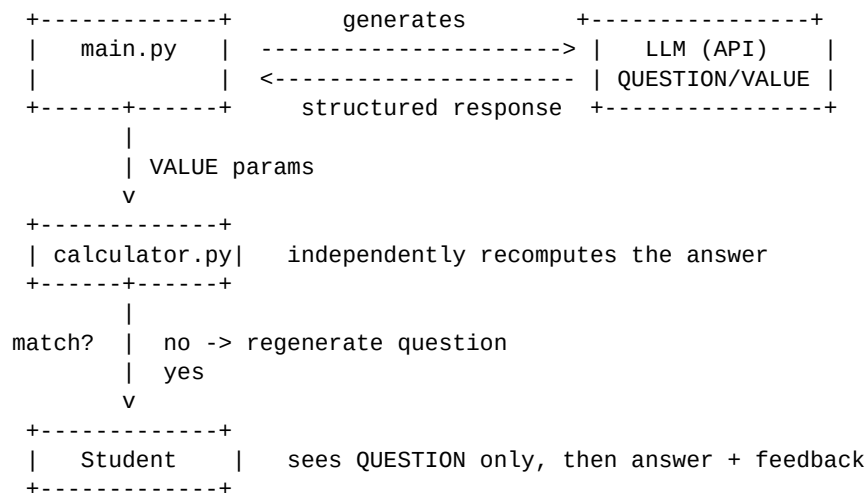
Putting all of the above together, a single question now flows through the system as follows:

- 1. main.py selects a topic and calls the LLM with the matching prompt template.
- 2. The LLM returns a response in the fixed QUESTION / VALUE / PERCENTAGE format.

- 3. main.py parses the structured fields out of the response.
- 4. The VALUE parameters are passed into the matching calculator.py function.
- 5. calculator.py's result is compared against the LLM's own claimed PERCENTAGE / answer.
- 6. If they disagree beyond tolerance, the question is discarded and step 1 repeats.
- 7. If they agree, only the QUESTION text is shown to the student; everything else is hidden.
- 8. The student submits an answer, which is checked against the verified value.
- 9. Feedback (correct/incorrect, verified answer, explanation) is shown.

19. Current Architecture Diagram

A simplified view of how the pieces connect:



20. Completed Work

Summarising what Version 2 has delivered so far:

- Verification architecture separating LLM generation from Python-based grading.
- calculator.py with tested functions for percentages, profit & loss, time & work, and averages.
- A structured QUESTION / VALUE / PERCENTAGE output format, with prompts engineered to hold it.
- Detection and regeneration of internally inconsistent (ambiguous) generated questions.
- Hidden-from-student handling of internal answer data until after submission.
- Tolerance-based numeric answer checking and a basic feedback message.
- An initial batch of successful end-to-end test runs across four categories.

21. Current Limitations

Version 2 fixes correctness, but is still narrow in scope and rough around several edges:

- calculator.py only covers a handful of aptitude categories so far — many common topics (ratios & proportions, simple/compound interest, time-speed-distance, permutations & combinations) are not yet verified.
- There is no persistence layer: nothing about a student's history, accuracy, or progress is stored between sessions.
- Difficulty is not adaptive — every question is generated at a similar difficulty level regardless of how the student is performing.
- The consistency check for ambiguous questions is still fairly simple and can miss subtler mismatches between phrasing and numbers.
- The interface is minimal, built around getting the pipeline correct rather than around presentation.

22. Next Milestone: progress.py

The next concrete milestone is progress.py, a module dedicated to tracking a student's performance over time instead of treating every question as a one-off, disconnected event.

- Log each attempt: topic, question, student answer, verified answer, correctness, timestamp.
- Maintain running accuracy per category, not just overall.
- Surface which categories are currently weakest, as a basis for future adaptive selection.
- Keep the same 'don't trust unverified numbers' discipline: progress stats are computed from calculator.py-verified results only, never from the LLM's own claims.

23. Memory and Adaptive-Learning Roadmap

Once progress.py exists, it becomes the foundation for a longer-term adaptive-learning roadmap:

- Short-term: bias topic selection toward the student's weakest categories rather than picking topics uniformly at random.
- Medium-term: introduce a simple spaced-repetition style scheduling, resurfacing a category again after a delay if recent accuracy in it was low.
- Longer-term: track difficulty per category and gradually increase it as accuracy improves, so the platform grows with the student instead of staying at one fixed level.
- All of this depends on the verification work already done in Version 2 — adaptive logic built on unverified LLM answers would just adapt to noise.

24. Remaining Features

Beyond the immediate roadmap, a longer backlog of features remains for later versions:

- Support for the remaining aptitude categories (ratios, interest, time-speed-distance, permutations & combinations, data interpretation).

- Richer explanations that walk through calculator.py's own intermediate steps, not just the LLM's narrative.
- A timer / test-mode that simulates real placement-test time pressure.
- A simple results dashboard summarising accuracy and time spent by category.
- Optional hints before a student gives up on a question.

25. Engineering Lessons Learned

A few lessons from building Version 2 that are worth carrying into everything that comes after it:

- An LLM's stated confidence in a number has no bearing on whether that number is correct — arithmetic needs a deterministic check, not a more convincing prompt.
- Structured, tightly-specified output formats are far more reliable to build on than free text, even if they take several prompt iterations to lock in.
- A failure that looks like 'the LLM made a small mistake' is often really two internally consistent but mutually incompatible pieces of information — worth checking for that specific pattern, not just re-verifying the arithmetic.
- Small, single-purpose, well-tested functions (like each one in calculator.py) are easier to trust and to extend than one large 'solve everything' function.
- Hiding internal/answer data from the user is easiest to get right when it's designed in from the start, rather than patched on after the interface already exists.

26. Current Milestone Status

As of this writing, the verification architecture, calculator.py, the structured output format, and basic feedback are complete and passing tests across the four supported categories. progress.py — the next milestone — has not yet been started; it is the immediate next step once the current verification pipeline is considered stable.

27. Conclusion

Version 2 does not change what AptitudeMind is trying to be — an unlimited, AI-generated aptitude practice tool for placement preparation. What it changes is whether that tool can be trusted. By refusing to let the language model be the final authority on any number, and instead routing every answer through a small, tested, deterministic Python module, Version 2 turns AptitudeMind from a demo that sometimes gets the math wrong into a system whose answers a student can actually rely on.

The categories covered are still limited and there is no memory of a student's progress yet, but the hardest architectural problem — how to keep LLM creativity without inheriting LLM arithmetic mistakes — is now solved and tested. Everything remaining, from progress.py onward, builds on top of that foundation rather than around it.